LAB 6 : DLQ5 - Learning to Bend a Model (Guided Workshop)

What This Session Is

This is a guided workshop, not a lab exam.

You will not be given finished code.

Instead:

- you will be told what needs to exist
- you will be given skeletons
- you will decide what goes inside
- if stuck, you may reason, discuss, or look up basic NumPy syntax only

No ML libraries.

No auto-differentiation.

No shortcuts.

The goal is to understand how learning is built, not to run a black box.

Big Picture Reminder (Before We Start)

So far, you have used models of the form:

$$\hat{y} = w^\top x + b$$

This creates a flat surface.

A “flat surface” means the model can only represent linear relationships. No matter how you tune the numbers, the shape does not curve.

Today, we want to allow the model to bend.

That means introducing:

- an intermediate computation
- a non-linear transformation
- a way to send error information backward

Step 0 — What You Are Allowed to Use

You may use:

- numpy
- basic Python loops and functions

You may not use:

- sklearn
- torch / tensorflow
- any automatic gradient tools

Step 1 — Create a Simple Dataset

- output increases early
- growth slows later

Task 1.1 — Input generation

You must create:

- one input feature
- values between 0 and 10
- total 50 samples

Skeleton:

```
# Create input data X
# Shape should be (50, 1)
# Hint:
```

```
    # - use np.linspace
    # - reshape properly
```

▼ Hint

X = np.linspace(0,10,50).reshape(-1,1)

Write a comment explaining:

- what X represents
- why this range was chosen

This range ensures we see both early rapid change and later saturation, which a linear model cannot fit well.

Task 1.2 — Target generation

The target should:

- increase non-linearly
- include small noise

Skeleton:

```
# Create target y
# Use a non-linear relationship
# Add small noise
# Constraints:
    # - noise should be small (not dominating)
    # - output should remain numeric
```

▼ Hint

```
noise = np.random.normal(0,0.2, size=(50,1))
y = np.log(X +1) + noise
```

Write a comment explaining:

- why this target cannot be fit well by a straight line

The logarithmic shape grows fast initially and slows later. A straight line has only one slope and cannot match both regions simultaneously.

Step 2 — Decide the Model Shape (No Code Yet)

Before writing any code, answer in comments:

- How many intermediate values (hidden units) do you want?

  choose 3

- Why more than 1?
- Why not too many?

You are fixing the shape of the model, not tuning numbers.

Hidden units allow multiple “pieces” of behavior. Too many units increase complexity and instability; too few cannot bend enough.

Step 3 — Initialize Model Parameters

You will need:

- weights from input → hidden
- bias for hidden
- weights from hidden → output
- bias for output

Constraints (important)

- weights must be initialized randomly in range [-1, 1]
- biases must start at 0

Skeleton:

```
# Initialize W1, b1
# Shapes must match input -> hidden
# Initialize W2, b2
# Shapes must match hidden -> output
# Use np.random.uniform
# Do NOT hardcode numbers
```

[Added code]

▼ Hint

```
W1 = np.random.uniform(-1,1, size=(1,3))
b1 = np.zeros((1,3))

W2 = np.random.uniform(-1,1, size=(3,1))
b2 = np.zeros((1,1))
```

Write comments explaining:

- what each parameter controls
- which ones affect shape vs position

Weights affect shape (slope and bending). Biases shift the curve up or down.

Step 4 — Define an Activation Function

You are not told what activation is yet.

You are told this:

> "We need a function that:

> - changes the shape of the model
> - has a slope
> - behaves differently for positive and negative values”

Task 4.1 — Write the function

Skeleton:

```
defactivation(z):
# Implement a function that:
    # - keeps positive values
    # - suppresses negative valuespass
```

▼ Hint

```
defactivation(z):return np.maximum(0, z)
```

Task 4.2 — Write its slope

Skeleton:

```
defactivation_slope(z):
# Return the slope of activation
# Slope must be:
# - zero in some region
# - non-zero elsewherepass
```

▼ Hint

```
defactivation_slope(z):return (z >0).astype(float)
```

Write a comment answering:

- why slope matters for learning

Without a slope, the model has no direction signal telling it how to change parameters.

Step 5 — Forward Pass (What the Model Does)

The forward pass computes predictions.

You must compute, in order:

$$z_1 = XW_1 + b_1$$

$$h = \text{activation}(z_1)$$

$$\hat{y} = hW_2 + b_2$$

Skeleton:

```
# Forward pass
# Step 1: linear transformation
# Step 2: activation
# Step 3: output prediction
```

▼ Hint

```
z1 = X @ W1 + b1
h = activation(z1)
y_hat = h @ W2 + b2
```

Write comments mapping each line to the math above.

Step 6 — Define Error and Loss

Task 6.1 — Error

Skeleton:

```
# Compute error
# error = prediction - actual
```

▼ Hint

```
error = y_hat - y
```

Task 6.2 — Loss

You must use Mean Squared Error.

$$L = \frac{1}{N} \sum (\hat{y} - y)^2$$

Skeleton:

```
# Compute loss
# Loss should be a single scalar
```

▼ Hint

```
loss = np.mean(error **2)
```

Write a comment explaining:

- why large errors matter more than small ones here

Squaring amplifies large mistakes, forcing the model to correct them more aggressively.

Step 7 — Why We Need Slopes (Conceptual Stop)

```
# To reduce loss, we must change parameters.
# To know how to change them, we need to know:
# if a small change increases or decreases loss.
```

This sentence is the reason gradients exist.

Step 8 — Backward Flow (Skeleton Only)

Task 8.1 — From loss to prediction

$$\frac{\partial L}{\partial \hat{y}} = 2(\hat{y} - y)$$

Skeleton:

```
# dL/dy_hat
# Hint: derivative of MSE
```

▼ Hint

```
dL_dy =2 * error /len(X)
```

Task 8.2 — Gradients of output layer

$$\frac{\partial \hat{y}}{\partial W_2} = h$$

$$\frac{\partial \hat{y}}{\partial b_2} = 1$$

Skeleton:

```
# dL/dW2
# dL/db2
```

▼ Hint

```
dL_dW2 = h.T @ dL_dy
```

```
dL_db2 = np.sum(dL_dy, axis=0, keepdims=True)
```

Write comments:

- why these depend on hidden values

Task 8.3 — Flow into hidden layer

$$\frac{\partial \hat{y}}{\partial h} = W_2$$

Skeleton:

```
# dL/dh
```

▼ Hint

```
dL_dh = dL_dy @ W2.T
```

Task 8.4 — Apply activation slope

$$\frac{\partial h}{\partial z_1} = \text{activation_slope}(z_1)$$

$$\frac{\partial L}{\partial z_1} = \frac{\partial L}{\partial h} \cdot \frac{\partial h}{\partial z_1}$$

Skeleton:

```
# dL/dz1 = dL/dh * activation_slope(z1)
```

▼ Hint

```
dL_dz1 = dL_dh * activation_slope(z1)
```

Write a comment explaining:

- how activation controls error flow

Task 8.5 — Gradients of first layer

$$\frac{\partial z_1}{\partial W_1} = X$$

$$\frac{\partial z_1}{\partial b_1} = 1$$

Skeleton:

```
# dL/dW1
# dL/db1
```

▼ Hint

```
dL_dW1 = X.T @ dL_dz1
dL_db1 = np.sum(dL_dz1, axis=0, keepdims=True)
```

Step 9 — Parameter Update Rule

$$\theta := \theta - \eta \frac{\partial L}{\partial \theta}$$

Skeleton:

```
# learning_rate = ?
# Update W1, b1, W2, b2
# subtract (learning_rate * gradient)
```

▼ Hint

```
learning_rate =0.01

W1 -= learning_rate * dL_dW1
b1 -= learning_rate * dL_db1
```

```
W2 -= learning_rate * dL_dW2
b2 -= learning_rate * dL_db2
```

Write a comment connecting this to:

- perceptron weight update
- “move opposite to error”

Step 10 — Training Loop (Big Skeleton)

Skeleton:

```
for epochinrange(?):# forward pass
# loss
# backward pass
# update parameters
# optionally print loss
```

▼ Hint

```
for epochinrange(1000):
    z1 = X @ W1 + b1
    h = activation(z1)
    y_hat = h @ W2 + b2

    error = y_hat - y
    loss = np.mean(error **2)

    dL_dy =2 * error /len(X)
    dL_dW2 = h.T @ dL_dy
    dL_db2 = np.sum(dL_dy, axis=0, keepdims=True)

    dL_dh = dL_dy @ W2.T
    dL_dz1 = dL_dh * activation_slope(z1)
    dL_dW1 = X.T @ dL_dz1
    dL_db1 = np.sum(dL_dz1, axis=0, keepdims=True)
```

```
    W1 -= learning_rate * dL_dW1
    b1 -= learning_rate * dL_db1
    W2 -= learning_rate * dL_dW2
    b2 -= learning_rate * dL_db2if epoch %100 ==0:print(ep
och, loss)
```

Decide:

- number of epochs
- learning rate

Justify both in comments.

What This Workshop Is Teaching You

By the end of DLQ5, you should understand:

- why linear models fail on curved data
- what “bending” actually means mathematically
- why activations exist
- why slopes are necessary
- how error flows backward
- that backpropagation is not magic, just organized slopes

Transition Line

“So far, learning only changed numbers.
Now, learning can change how patterns are represented.
Next, we will compare different ways of bending.”